

Lab 2: Introduction to Linux (with focus on Kali)

Part 1: Exploring the System

Objective: Become familiar with the file system structure and sensitive files.

1. Navigate from `/` to `/etc`, `/var`, and `/home`. Identify the purpose of each.
2. Read the contents of `/etc/passwd` and answer:
 - What type of information does it contain?
 - What is the difference between this file and `/etc/shadow`?
3. Check the permissions of `/etc/shadow` with `ls -l` and explain why it cannot be read as a normal user.
4. Review logs with:

```
less /var/log/auth.log
less /var/log/syslog
```

Pair discussion: What types of security events can be detected in these logs?

Part 2: Files and Permissions

Objective: Understand permissions, ownership, and file management.

1. Create a working folder in `~/linux_practice`.
2. Create three files with fictitious content.
3. Use the following commands and document their effects:

```
cp file1.txt copy.txt
mv file2.txt renamed.txt
rm file3.txt
chmod 644 copy.txt
chmod +x renamed.txt
chown root:root copy.txt
```

Collaborative activity: In pairs, design an example where misuse of `chmod` or `chown` causes a security problem.

Part 3: Processes and Services

Objective: Explore system processes and service management.

1. Run `top` and observe the most active processes. Filter with `ps aux | grep bash`.
2. Start a program (for example, `mousepad` or `ping 8.8.8.8`) in the background and kill it with `kill`.
3. Check the status of `ssh`, `cron`, and `cups`:

```
systemctl status ssh
service cron status
```

Reflection: Why should some services be active while others should not in a secure environment?

Part 4: Package Management

Objective: Learn how to install and manage software.

1. Search for a package such as `nmap`, `htop`, or `net-tools` using:

```
apt search nmap
```

2. Install one of those packages, verify its installation, and then uninstall it.

Discussion: What security risks are involved in installing packages from untrusted sources?

Part 5: Automation with Bash

Objective: Write a simple script to automate tasks.

1. Create a script that:
 - Finds all `.txt` files in the home directory.
 - Counts the lines of each file.
 - Prints the result ordered by number of lines.
2. Add improvements to the script:
 - Save the results to a file.
 - Accept as argument the base directory to scan.

Optional challenge: Modify the script to ignore files with more than 500 lines.

Submission

Each pair must submit a compressed file (`.zip` or `.tar.gz`) with:

- Key screenshots.
- The Bash script with comments.
- A document with answers to questions and reflections (can be `.md` or `.pdf`).